# Free Value Or Else

Steve Downey <sdowney@gmail.com>
Corentin Jabot <corentin.jabot@gmail.com>

| | |
|---|---|
| Document #: | D4270R1 |
| Date: | 2026-10-23 |
| Project: | Programming Language C++ |
| Audience: | LEWG |

**Abstract**

This paper proposes the member functions `value_or_construct` and `value_or_else` of [P3413R0] on `std::optional` and `std::expected`, which build the alternative only when it is needed, and the free functions `value_or`, `reference_or`, `or_invoke`, and `or_construct` over any type modeling a `nullable` concept, with the result type computed as the `common_type` or `common_reference` of the contained value and the alternative. The two were separate papers until LEWG asked for one.

## Contents

**1 Comparison table** **2**

**2 Motivation** **3**

**3 Design** **4**

**4 P3413R0, and the two halves** **9**

**5 Principles for Reification of Design** **10**

**6 Implementation Experience** **10**

**7 Proposal** **13**

**8 Wording** **13**

**9 Impact on the standard** **17**

**10 Acknowledgments** **17**

**11 Document history** **17**

**References** **18**

## Changes Since Last Version

— **R1**

— Unified with [P3413R0] at LEWG's request: Corentin Jabot joins as co-author, and the members `value_or_construct` and `value_or_else` are folded into the proposal, the wording, and the implementation, covering `optional<T&>`, which P3413R0 deferred and which is now C++26.

- An rvalue nullable moves its payload when its dereference expression permits moving. R0 dereferenced an lvalue always, observing and never consuming; the models now split by dereference behavior, so `optional` and `expected` move, while raw and smart pointers copy.
- `nullable` is proposed as a named, public concept. R0 specified it as exposition-only and asked LEWG for a poll.
- The concept is stated on `const remove_reference_t<T>&` rather than `const T`, so the const-observability requirement holds for the deduced `T` of an lvalue argument, where the const was silently discarded before.
- `reference_or`'s second dangling guard is stated on the dereference type rather than on `T&`, where it could never fire.
- `reference_or` mandates both that its result is a reference and that an rvalue argument is a `borrowed_nullable`. Temporary owning wrappers are therefore rejected even when dereferencing them yields `T&`.
- `or_invoke` forwards its invocable, agreeing with the `invoke_result_t<I>` its return type is computed from; `or_construct` gains the engaged-path *Mandates* and rejects reference result types.
- The asks to LEWG are cut from nine to zero. The dangling and borrowing checks are *Mandates*, the concept stays minimal, the names stand, and the header is `<optional>`.
- Wording rebased to [N5054], which carries `optional<T&>`.

— **R0** Initial revision.

# 1 Comparison table

## 1.1 Raw pointer with a manual null check

The fundamental nullable type is the raw pointer, and the fundamental idiom for providing a default is the conditional expression guarded by a null check. The free function gives that pattern a name, removes the repetition of the checked entity, and computes the type of the result correctly.

```
Cat* cat = find_cat("Fido");
return cat ? *cat : Cat{"Default"};
```

```
Cat* cat = find_cat("Fido");
return std::value_or(cat, Cat{"Default"});
```

## 1.2 The fallback that is built anyway

The argument to `value_or` is an ordinary function argument, constructed on every call and destroyed unused on every engaged one. The member `value_or_construct` takes the fallback's constructor arguments instead, and builds nothing when the optional is engaged; `value_or_else` defers an arbitrary computation the same way.

```
std::optional<Config> o = load();
// Config built even when o is engaged
return o.value_or(Config{"/etc/default"});
```

```
std::optional<Config> o = load();
// built only when o is disengaged
return o.value_or_construct("/etc/default");
```

## 1.3 The `value_or` truncation pitfall

Because `std::optional<T>::value_or` converts its argument to `T`, a perfectly reasonable-looking sentinel silently truncates. The free function computes the common type of the contained value and the alternative, and converts both *outward* to that type instead of forcing the alternative *inward* through `T`.

```
std::optional<std::uint16_t> o = get();
auto v = o.value_or(-1);
// v is uint16_t{65535}
```

```
std::optional<std::uint16_t> o = get();
auto v = std::value_or(o, -1);
// v is int; -1 is preserved
```

## 1.4 Reference alternative without copies

When both the contained value and the alternative are references to long-lived objects, the result should be a reference. Member `value_or` cannot do this; `reference_or` can, and statically rejects the cases where the result would dangle.

```
std::optional<Logger&> opt = get_logger();
Logger& l =
    opt ? *opt : default_logger();
```

```
std::optional<Logger&> opt = get_logger();
Logger& l = std::reference_or(
    opt, default_logger());
```

## 2 Motivation

`std::optional<T>::value_or` answers the most common question asked of an optional — "the value, or this instead" — and gets two things wrong. It answers with the wrong type, funneling both the contained value and the alternative through `T`. And it pays for the alternative whether or not it is wanted, because the alternative is an ordinary function argument. The two defects are independent, they have been pursued independently, and this paper carries the fix for each.

### 2.1 The type of the answer

`value_or` returns `T`, and it requires the alternative to be convertible to `T`. Every use therefore funnels both the contained value and the alternative through `T`, whether or not `T` is the right type for the result. The consequences are familiar: sentinels truncate (`optional<uint16_t>{}.value_or(-1)` is `65535`), wider alternatives narrow, and reference results are simply impossible, so `value_or` forces a copy even when both candidate results are references to long-lived objects.

This is not fixable in place. Changing the return type of a member function that has been shipping since C++17 is both an API and an ABI break, and the &&-qualified overload set compounds the problem. During the work on `std::optional<T&>` [P2988R12] the question was reopened in earnest, because for an optional reference the cost of the wrong answer is no longer a pessimization but a wrong program: returning `T` from `optional<T&>::value_or` copies, returning `T&` dangles when the alternative is a temporary. After extensive discussion in LEWG, P2988 concluded that "there is no particularly wonderful solution for `value_or` that does not involve a time machine," adopted the least objectionable member behavior — return `remove_cv_t<T>` by value — and explicitly forecast this paper: free functions over all types modeling an optional-like concept, where the return type can finally be computed correctly.

Existing practice diverges because there is no one member-function answer that satisfies everyone:

| Implementation | Behavior |
|---|---|
| Standard | `optional<T>::value_or` returns a `T` |
| Boost | `optional<T>::value_or` returns a `T` |
| | `optional<T&>::value_or` returns a `T&` |
| TartanLlama | `optional<T>::value_or` returns a `T` |
| | `optional<T&>::value_or` returns a `T` |
| Flux | returns the type of the ternary, similar to `common_reference` |
| Think-Cell | returns `common_reference`, with some caveats |

The tools to compute the right answer postdate `optional`. `std::common_type` existed but was not used; `std::common_reference` arrived with ranges in C++20; and `std::reference_constructs_from_temporary_v` [P2255R2] arrived in C++23, making it possible for the first time to *statically reject* the dangling cases that made reference-returning `value_or` untenable. The conditional expression `b ? *m : u` has computed the common type all along; the library can now name that computation, add the lifetime check the core language does not perform, and do so once, for every nullable type.

That last point is the generalization this paper takes from [P1255R13]: being "contextually convertible to `bool` and dereferenceable" is a protocol. Raw pointers, `std::shared_ptr`, `std::unique_ptr`, `std::expected<T,E>`, and `std::optional<T&>` all speak it, as `std::optional` does. Each of them today requires either a hand-written conditional or a type-specific spelling of "the value, or this instead." A single constrained free function family covers them all, with one set of semantics to learn and one place to get the corner cases right.

### 2.2 The cost of the alternative

The second defect is independent of the first, and older in the committee's record. Passing a `Config` as the alternative constructs it on every call and destroys it unused on every engaged one. The parameter is an ordinary function parameter, so the alternative is evaluated whether or not it is reached, and the cost is unbounded: an expensive construction, an allocation, a lookup, a computation whose result is thrown away. The workaround is the conditional expression the member was supposed to replace, and callers who care about the cost write it.

The cure has been proposed before. [P2218R0], by Marc Mutz, proposed `value_or_construct`, taking the fallback's constructor arguments rather than a constructed fallback, so that nothing is built on the engaged path; it did not proceed at the time. [P3413R0] revived it, added `value_or_else` for the general case of a

lazily evaluated nullary callable, and specified both on `std::optional` and `std::expected`. Those members are the member half of this paper, and their wording is reproduced below.

Laziness is orthogonal to the type of the result. `value_or_construct` and `value_or_else` still return `T`, and still require the alternative to converge on `T`. They fix what they set out to fix, on the two types that have the member to grow.

### 2.3 Two defects, two shapes

The fixes take different shapes because the defects are of different kinds. The cost is a defect in the parameters, and a new member name fixes it in place: `value_or_construct` and `value_or_else` sit beside `value_or` on `optional` and `expected`, spelled the way the member they join is spelled, and nothing existing changes meaning. The type is a defect in the signature, and no member can fix it. The return type of a member shipping since C++17 can not change. A corrected second member would have to coexist with the first forever, written and kept in sync by hand on every class that has members at all, including each smart pointer. A raw pointer has none. A constrained free function template states the semantics once and applies them to every type speaking the protocol, including user-defined nullables, which model `nullable` without doing anything to opt in.

So this paper proposes each defect's fix in the shape that defect admits: a pair of members, and a family of free functions over the protocol. Neither forecloses the other, and neither depends on the other.

## 3 Design

The proposal has two halves. The free half is four function templates over a `nullable` concept:

```
template <class T>
concept nullable = requires(const std::remove_reference_t<T>& t) {
    bool(t);
    *(t);
};

template <class T>
inline constexpr bool enable_borrowed_nullable = false;

template <class T>
concept borrowed_nullable =
  nullable<T> &&
  (std::is_lvalue_reference_v<T> ||
   enable_borrowed_nullable<std::remove_cvref_t<T>>);

template <class T>
inline constexpr bool enable_borrowed_nullable<T*> = true;

template <class T>
inline constexpr bool
  enable_borrowed_nullable<std::optional<T&>> = true;

template <class T, class E>
inline constexpr bool
  enable_borrowed_nullable<std::expected<T&, E>> = true;

// The type *m yields, carrying the value category of the nullable through
// to its payload.  Exposition only in the wording.
template <class T>
using deref_t = decltype(*std::declval<T>());

template <nullable T, class U,
          class R = std::common_reference_t<deref_t<T>, U&&>>
constexpr auto reference_or(T&& m, U&& u) -> R;

template <nullable T, class U,
          class R = std::common_type_t<deref_t<T>, U&&>>
constexpr auto value_or(T&& m, U&& u) -> R;
```

```
template <nullable T, class I,
          class R = std::common_type_t<deref_t<T>,
                                       std::invoke_result_t<I>>>
constexpr auto or_invoke(T&& m, I&& invocable) -> R;

template <class Ret = void, nullable T, class... Args,
          class R = std::conditional_t<std::is_void_v<Ret>,
                        std::remove_cvref_t<deref_t<T>>, Ret>>
constexpr R or_construct(T&& m, Args&&... args);

template <class Ret = void, nullable T, class E, class... Args,
          class R = std::conditional_t<std::is_void_v<Ret>,
                        std::remove_cvref_t<deref_t<T>>, Ret>>
constexpr R or_construct(T&& m, std::initializer_list<E> il, Args&&... args);
```

All four have the same engaged-path behavior, producing `*std::forward<T>(m)` converted to `R`, and differ in how the alternative is supplied (a value, a value bound as a reference, a nullary invocable, constructor arguments) and in how `R` is computed (`common_type`, `common_reference`, or the element type itself).

The member half is `value_or_construct` and `value_or_else` on `std::optional` and `std::expected`, as [P3413R0] specified them:

```
template <class... Args>
constexpr T value_or_construct(Args&&... args) const&;

template <class U, class... Args>
constexpr T value_or_construct(std::initializer_list<U> il,
                               Args&&... args) const&;

template <class F>
constexpr T value_or_else(F&& f) const&;

// and the corresponding && overloads, on optional and expected alike
```

Both return `T`, as the member `value_or` beside them does, and neither constructs anything on the engaged path.

### 3.1 The `nullable` concept

The concept is the "maybe protocol" of [P1255R13]: an object is nullable if it is contextually convertible to `bool` and dereferenceable. The requirements are stated on a const lvalue: observation must not require mutable access. The reference is stripped before the const is applied, so the rule holds for the deduced `T` of an lvalue argument as much as for an rvalue; spelled `const T` it would not, since const applied to a reference type is discarded. A type whose `operator bool` or `operator*` is non-const does not model `nullable`. Asking "the value or a default" is a query. A type that can not be queried through const access is not answering it.

Types verified to model the concept: `std::optional<T>`, `std::expected<T,E>`, raw object pointers `T*`, `std::shared_ptr<T>`, `std::unique_ptr<T>`, and `std::optional<T&>` as specified by [P2988R12]. Types verified not to model it: arithmetic types and `bool` (contextually convertible to `bool` but not dereferenceable), `std::string` and the containers (neither), `std::nullptr_t`, `void`, and types providing only one of the two operations. Iterators in general do not model `nullable` because they are not contextually convertible to `bool`. Correctly so: an iterator's validity is not a property the iterator itself can report.

One possible refinement is not taken. P1255's `nullable_object` also requires that the result of dereferencing be an equality-preserving *object*, which excludes function pointers; the minimal concept above admits them (a function pointer is contextually convertible to `bool` and dereferenceable), and they are then rejected only downstream, when `common_type` fails. We keep the concept minimal. Requiring `is_object_v` of the referent would move that rejection to the constraint, which reads better in a diagnostic, at the cost of a concept users can no longer reproduce from its one-line statement.

### 3.2 Borrowed nullables

`nullable` says that an object can report whether dereferencing it is valid. It does not say that the result of dereferencing survives destruction of the nullable itself. That distinction does not matter to the three value-producing functions, but it is decisive for `reference_or`.

A nullable can be understood as a view of zero or one element. In a `borrowed_nullable`, it is that element — the result of dereferencing — that is borrowed: its lifetime is managed independently of the nullable wrapper. The name does not mean that the nullable object itself is borrowed.

This is the same lifetime distinction made by ranges. A `single_view<T>` contains its element, and destroying the view invalidates references to that element; an `optional<T>` has the same relationship to its value. A `span<T>` presents externally owned elements, as a raw pointer, `optional<T&>`, or `expected<T&, E>` presents an externally owned referent. `borrowed_range` permits an iterator or reference obtained from an rvalue range to escape the range wrapper; `borrowed_nullable` permits the reference obtained by dereferencing an rvalue nullable to escape through `reference_or`.

As with `borrowed_range`, an lvalue nullable is always safe to observe: the caller's object remains alive after the call. An rvalue must opt in via `enable_borrowed_nullable`. Raw pointers opt in because destroying a pointer does not destroy its pointee. `optional<T&>` and `expected<T&, E>` opt in because they store references and destroying the wrapper does not destroy the referred-to object.

Owning `optional` and `expected` do not opt in. Neither do `unique_ptr` or `shared_ptr`: although their dereference operators return `T&`, a temporary `unique_ptr`, or a temporary `shared_ptr` holding the last ownership share, destroys the referent at the end of the full-expression. A custom deleter that happens not to delete is not the lifetime guarantee on which a generic reference-returning function may rely. Users may still pass every one of these types as an lvalue.

This rule has an ergonomic cost for an owning nullable returned by value. Even if the selected reference would be consumed immediately, the temporary wrapper cannot promise that its referent survives the return:

```
reference_or(lookup(key), fallback); // ill-formed, even if consumed immediately

auto result = lookup(key);
reference_or(result, fallback);      // the wrapper remains alive

value_or(lookup(key), fallback);     // produce a value instead
```

A program-defined nullable may specialize `enable_borrowed_nullable` to `true` when destruction of the nullable cannot invalidate the result of dereferencing it.

### 3.3 Exposition-only, or a named public concept?

We propose `std::nullable` as a named, public concept, usable outside the standard library.

The protocol is vocabulary. [P2988R12] forecast these functions "over all types modeling optional-like, `concept std::maybe`", and [P1255R13] is titled, in part, "a concept to constrain maybes" — the concept has been treated as a deliverable in its own right throughout this line of work. Users writing their own functions over nullable types (a `value_or` variant with different policy, an algorithm taking a range of nullables) need the same constraint, and without a public name each codebase re-derives it, and not always the same way. A named concept can be tested directly (`static_assert(std::nullable<T>)`), appears by name in diagnostics, and fixes the meaning of "optional-like" across the ecosystem once.

There is a principled reason not to. A named standard concept is forever, and a public name can never be demoted or meaningfully tightened, while an exposition-only one can be promoted later without breaking anything. The library has a directly analogous precedent: *`boolean-testable`*, among the most-used constraints in the standard, is exposition-only for that reason. However, the asymmetry cuts the other way here. A constraint nobody can name is one every codebase re-derives, and the re-derivations are what diverge; *`boolean-testable`* is plumbing for other concepts, where `nullable` is the thing users want to write down. With [P1255R13] concluded (see the discussion of `yield_if` below), this paper is the concept's sole remaining home, so there is no cross-paper coordination to wait on.

The semantics a public name has to carry are stated with it: the contextual conversion is equality-preserving and does not modify the object; `*t` may be evaluated only when that conversion yields `true`; and both are valid on a const lvalue. Making the concept exposition-only instead would cost nothing but the spelling, but this paper proposes the public name.

### 3.4 Return type computation

`deref_t<T>` is `decltype(*declval<T>())`, the type `*m` yields, with the value category of the nullable carried through. For a nullable passed as an lvalue with an `int` payload it is `int&`; for a `const optional<int>` it is `const int&`; for `const int*` it is `const int&`; and for an rvalue `optional<int>` it is `int&&`. The return type is then the common type (for `value_or` and `or_invoke`) or the common reference (for `reference_or`) of that dereference type and the alternative.

For `value_or`, `common_type` decays, so the result is a prvalue, and mixed-type uses promote instead of truncating:

| nullable `T` | alternative `U` | result `R` |
|---|---|---|
| `optional<int>` | `int` | `int` |
| `expected<int,E>`, `int*`, `shared_ptr<int>`, `unique_ptr<int>` | `int` | `int` |
| `optional<int>` | `long` | `long` |
| `optional<int>` | `double` | `double` |
| `optional<uint16_t>` | `int` (e.g. `-1`) | `int` |
| `optional<string>` | `string` | `string` |

`value_or(m, u)` is the library spelling of `m ? *m : u`; the member is `m ? *m : T(u)`. The result has the type the first conditional expression produces.

### 3.5 Value categories, and what an rvalue nullable means

The value category of the nullable reaches its payload. `deref_t<T>` is `decltype(*declval<T>())`, so an expiring nullable dereferences to an expiring payload, and the engaged path moves rather than copies: `value_or(std::move(opt), u)` matches `std::move(opt).value_or(u)`. The free functions do not lose the member's efficiency.

`std::optional` and `std::expected` contain their value. An rvalue of one is an expiring object whose payload expires with it, and `operator*` is ref-qualified to say so. A pointer contains nothing. `T*`, `std::shared_ptr`, and `std::unique_ptr` dereference to `T&` even as rvalues, and therefore the value-producing functions copy their referents. This rule follows the expression's value category; it makes no independent ownership inference.

`std::optional<T&>` settles which property is doing the work. It is an `optional`, and it dereferences to `T&`. Each nullable reports what its own value category permits the value-producing functions to do with its payload.

The alternative is to compute the dereference type from `T&` always, as `std::iter_reference_t` does, so that the functions observe and never consume. It buys a rule that fits in a sentence: the result is what the lvalue conditional expression would have produced. And two smaller things. The return type of the value-producing functions stops depending on the value category of the nullable, and there is no moved-from path to reason about. However, it buys them by making every owning nullable copy a payload that is going away, which is the wrong answer stated uniformly. We would rather be correct and pay the ergonomics.

The behavior is pinned either way. The engaged path is copy-counted for `optional`, `expected`, and both smart pointers, so the dereference result shows up as test output and a change of rule shows up as a test failure (see Implementation Experience). This does not govern `reference_or`; that operation separately requires `borrowed_nullable`.

### 3.6 `reference_or`

`reference_or` accepts a `nullable` and mandates that it is a `borrowed_nullable`, computes `common_reference_t<deref_t<T>, U&&>`, and binds the result to the contained value or to the alternative without copying either. Const qualification flows through the computation as it should: a `const optional<int>` or a `const int&` alternative each force `const int&` as the result; an `int*` with an `int&` alternative yields `int&`, writable through, and mutation through the result reaches the original object. The function introduces no copies.

The common reference of two unrelated lvalue references need not be a reference. `common_reference_t<int&, long&>` is `long`, and `common_reference_t<string&, const char(&)[2]>` is `string`; in such calls the common-reference computation creates a result object rather than selecting either source object. `reference_or` therefore mandates that its result type is a reference. Calls whose common reference is a value are ill-formed and should use `value_or`; a function named for reference semantics does not silently copy.

The danger of returning references is binding one to a temporary, and that is now statically detectable. The implementation carries two guards:

```
static_assert(!std::reference_constructs_from_temporary_v<R, U>);
static_assert(
    !std::reference_constructs_from_temporary_v<R, deref_t<T>>);
```

These checks complement, rather than replace, `borrowed_nullable`. The first rejects the alternative materializing a temporary: in `reference_or(opt, 0)` the computed common reference of `int&` and `int&&` is `const int&`, which would bind to a temporary materialized from the literal, and the call is ill-formed. The second rejects the symmetric case where the conversion of `*m` to `R` would materialize a temporary. The mandate rejects an rvalue wrapper whose destruction may invalidate an otherwise direct reference. Together the three checks reject the known ways this operation could return a dangling reference, including temporary owning smart pointers whose `operator*` itself only reveals `T&`.

These checks are specified as *Mandates*, implemented by `static_asserts`. A dangling call is therefore a hard, clearly-diagnosed error wherever it occurs, including inside an overload-resolution context. Specified as constraints, they would make the overload disappear SFINAE-style, which composes better with generic code probing for usability but turns a lifetime bug into a silent "no matching function" or, worse, a fallback to a different overload. Our position is that a dangling `reference_or` call is a bug, and should be reported as one.

The Tokyo discussion of the P2988 homework concluded that `value_or` is not an available name for a reference-returning function, and suggested `ref_or`. This paper spells it `reference_or`, matching the library's general preference for unabbreviated names.

### 3.7 `or_invoke` and laziness

`value_or` and `reference_or` evaluate their alternative unconditionally. When it is expensive to construct, that cost is unwelcome on the engaged path, and the established cure is to pass a nullary invocable that is called only when needed. `or_invoke(m, f)` returns `*m` converted to `R` when `m` is engaged and `f()` converted to `R` otherwise, with `R = common_type_t<deref_t<T>, invoke_result_t<I>>`; the invocable is not called on the engaged path.

Because `or_invoke` uses `common_type`, an invocable returning a reference still produces a value: `common_type_t<int&, int&>` is `int`. The family therefore covers three of four quadrants, eager value (`value_or`), eager reference (`reference_or`), and lazy value (`or_invoke`, and `or_construct` below), and omits the fourth. A lazy reference-returning form is specifiable (the dangling guard would apply to `invoke_result_t<I>` just as it does to `U`), but the use case, a reference alternative expensive to *name* and not to construct, is thin, and it is not proposed. That quadrant could compose naturally as a separate `reference_invoke` proposal later.

The obvious name, `or_else`, is taken: `std::optional::or_else` returns an `optional`, re-wrapping the result, and giving the same name to a value-producing free function would make `x.or_else(f)` and `or_else(x, f)` mean different things. The member half faces the same collision, and answers it as [P3413R0] did, with `value_or_else`. For the free function this paper proposes `or_invoke`. The two names differ because the two functions differ: the member returns `T`, the free function the common type.

### 3.8 `or_construct`: the fallback as constructor arguments

`or_construct` is the free-function analogue of the other half's member `value_or_construct`: the alternative is supplied not as a value but as constructor *arguments*, and the fallback object is constructed lazily, only on the disengaged path. On the engaged path nothing is constructed from them at all. An `initializer_list` overload provides the same convenience the member form has.

However, its return-type rule differs from the other three. There is no fallback *value* whose type could participate in a `common_type` computation, only arguments waiting to become one, so by default `R` is the decayed element type itself, `remove_cvref_t<deref_t<T>>`, and the conversion runs *inward*: the arguments construct an `R` directly, and the engaged payload converts to `R`. This is the member function's model, and here it is the right one; the outward common-type computation is meaningless without a second value.

What the free function can offer that the member cannot is choice of the constructed type. The leading `Ret` template parameter (defaulted through a `void` sentinel) lets the caller name the result type explicitly:

```
std::optional<std::pmr::string> m = lookup(key);
auto s = std::or_construct<std::string>(m, "default");
// std::string either way: converted from the payload, or
```

```
// constructed from the arguments --- chosen by the caller
```

The fallback is direct-initialized, so `explicit` constructors are usable; the arguments were always explicitly a construction request. Left to its default, `R` is a decayed object type, so `or_construct` returns a prvalue and can not dangle. With `optional<T&>` the result is a value copy of the referent or a freshly constructed object, never a reference. An explicitly named `Ret` is likewise required not to be a reference, because otherwise fallback construction could bind the returned reference to a temporary.

### 3.9 The member half: `value_or_construct` and `value_or_else`

The members are as [P3413R0] specified them, and their design questions are settled by the member they join. Both return `T`, and require the alternative to converge on `T`, because a member named for `value_or`, sitting beside it, would be a trap if it answered with a different type; the corrected result type is the free half's business, under its own names. Both come in `const&` and `&&` form, so an rvalue moves its payload out. `value_or_construct` takes constructor arguments, with an `initializer_list` overload, and direct-initializes the fallback, as `or_construct` does. `value_or_else` takes a nullary invocable whose result must convert to `T`. Neither touches the fallback on the engaged path. `expected<void,E>` gains neither; there is no value to construct.

The names do not collide: `x.value_or_construct(args...)` and `or_construct<R>(m, args...)` are different spellings of the same model, and `x.value_or_else(f)` and `or_invoke(x, f)` differ in return type (see `or_invoke` and laziness, above). And `optional<T&>` no longer has to be deferred: P3413 deferred it to [P2988R12] while that paper was in flight, and `optional<T&>` is in C++26 now, so the members are specified over it here, returning a value, exactly as its `value_or` does. `expected<T&,E>` is the one form still waiting on a paper of its own, and the wording says so.

### 3.10 Why not customization point objects

These are ordinary function templates rather than customization point objects. There is nothing to customize: the semantics are fully determined by the protocol, and a type that wants different behavior for "value or default" wants something other than `nullable`. The usual argument for a CPO, dispatching to a type's own implementation, is therefore absent, and the names do not inhibit ADL.

### 3.11 `std::expected`, and the error

`std::expected<T,E>` models `nullable`: both `bool(e)` and `*e` are valid on a const `expected`. Consequently `value_or(e, u)` works, and discards the error, as the member `value_or` does. This is by design: the moment the question is "the value, or this instead," the error has been decided to be uninteresting. Uses that need the error have `error_or` and the monadic interface on the member side; no error-observing overload is proposed here.

### 3.12 What about `yield_if`, and the end of P1255

[P2988R12] forecast a fourth function, `yield_if`, producing a view of zero or one elements, alongside the three proposed here. It was to remain with the [P1255R13] ranges work. However, that work is concluded. Range support for `std::optional` [P3168R2] supplanted `views::maybe`, and `views::nullable` with it: an optional now *is* the view of zero or one elements, and a pointer-like nullable reaches the same place through `optional<T&>` [P2988R12], as `p ? optional<T&>(*p) : nullopt`. Downey, its author, does not expect to revise P1255 further; this paper inherits its remaining deliverables, the concept and the free functions over it.

That covers `yield_if` as well: `yield_if(cond, value)` is now spellable as `cond ? optional(value) : nullopt`, which is itself a range. What residual demand exists is for a named convenience in comprehension-style pipelines; if implementation experience shows it is wanted, it is a small separate paper. It is explicitly not part of this proposal.

## 4 P3413R0, and the two halves

[P3413R0] ("A more flexible `optional::value_or` (else!)"), by Corentin Jabot, reviving [P2218R0], was a separate paper. It proposed the member `value_or_construct`, constructing the fallback `T` lazily and in place from forwarded arguments, with `initializer_list` overloads, and `value_or_else`, taking a lazily evaluated nullary callable, on both `std::optional` and `std::expected`. Reviewing R0 of this paper in Brno, LEWG saw the two as fixes to the same member function and asked, "Can we ask you and Corentin to come

back with one proposal?" This is that proposal. Corentin is a co-author here, and P3413's members, wording, and rationale are the member half of this document; P3413R0 is not pursued separately.

Unification changed presentation, not design. The members are as P3413 specified them, with one substantive addition: they are specified over `optional<T&>`, which P3413 deferred and which is now in C++26 (see The member half). The free half is untouched by the unification; where it has changed since R0, in the rvalue rule of Design, it changed for its own reasons.

The member half fixes the *cost* of the alternative and leaves the type alone: `optional` and `expected` each grow six functions, all returning `T`, and pointers of any kind get nothing. The free half fixes the *type* of the answer and its *reach*. Side by side:

| | member half (P3413R0) | free half |
|---|---|---|
| form | members | free functions |
| applies to | `optional`, `expected` | any `nullable` (incl. pointers) |
| return type | `T` | `common_type` / `common_reference` |
| truncation pitfall | unchanged, by design | fixed |
| reference results | no | `reference_or`, dangling-checked |
| lazy alternative | `value_or_else` | `or_invoke` |
| in-place fallback construction | `value_or_construct` | `or_construct`, caller-chosen result type |
| rvalue nullable moves payload | yes (`&&` overloads) | yes, for the owning models (see Design) |
| `optional<T&>` | members return a value | covered by the concept |

The halves compose, and they remain severable: LEWG can take one and leave the other, and each stands alone.

## 5 Principles for Reification of Design

The same principles that governed [P2988R12] govern here.

**Errors that can be detected at compile time must be detected at compile time.** The concept rejects non-nullable arguments at the constraint; `reference_or` additionally mandates against rvalue wrappers that do not model `borrowed_nullable`, non-reference common-reference results, and every conversion that would bind the result to a temporary.

**Never knowingly produce a dangling reference.** The functions take no address and extend no lifetime; `reference_or`'s result always refers to an object that existed before the call. The second guard is stated on `deref_t<T>`, the type `*m` yields; stated on `T&` it would name the nullable rather than its payload, and could never fire. The rejected cases are those where the core language would have materialized a temporary: a prvalue alternative whose common reference with the dereference type is a const reference (`reference_or(opt, 0)`), and conversions of the contained value to the result type that materialize. The `borrowed_nullable` mandate covers the orthogonal danger that destroying a temporary nullable destroys its directly referenced payload.

**Value-producing functions never dangle by construction.** `value_or`, `or_invoke`, and `or_construct` return non-reference values; `or_construct` mandates this even when its result type is explicitly selected.

These principles are enforced by a negative-compilation test suite in the reference implementation: each forbidden combination is a translation unit that must fail to compile, with the diagnostic matched against the constraint or `static_assert` that is required to fire.

## 6 Implementation Experience

The reference implementation is `beman.free_value_or` [Downey_free_value_or]. The implementation floor is C++23, required for `reference_constructs_from_temporary_v`; the functions are otherwise C++20 material.

The test suite exercises a matrix:

— `std::optional<T>`, `std::expected<T,E>`, `T*`, `std::shared_ptr<T>`, and `std::unique_ptr<T>` as the nullable; and, gated on a C++26 toolchain via the Beman implementations, `std::optional<T&>` and `std::expected<T&,E>`;

— engaged and disengaged;

— the nullable as lvalue, const lvalue, and rvalue, the rvalue cases copy- and move-counted on the engaged path, so that dereference behavior is pinned rather than assumed;

— the alternative as lvalue and rvalue, and mixed-type combinations;

— constant evaluation of all four functions;

— laziness of `or_invoke` and `or_construct`, and eagerness of `value_or`;

— for `or_construct`, construction arities from zero arguments through multi-argument emplacement and the `initializer_list` overload, with construction confirmed to occur only on the disengaged path; and

— negative-compilation cases for concept violations, non-reference common-reference results, conversion-created temporaries, and temporary owning `optional`, `expected`, `unique_ptr`, and `shared_ptr` wrappers; and for `or_construct`, an explicitly selected reference result, a payload that cannot be explicitly converted, and fallback arguments that cannot construct the result.

Reference identity and mutation round-trips confirm that `reference_or` introduces no copies: writes through its result reach the contained object when engaged and the alternative when not. The `optional<T&>` suite additionally verifies rebinding semantics: after an `optional<T&>` is rebound to a new referent, `value_or` produces and `reference_or` refers to the new referent, and disengaging falls through to the alternative.

The suite found two behaviors reported in Design: that a non-reference common-reference result must be rejected, and that `or_invoke`'s use of `common_type` decays reference-returning invocables to values, leaving the lazy-reference quadrant of the family empty by choice. The negative-compilation harness was itself validated: each WILL-FAIL translation unit was checked to fail for the required reason by mutating its expected-diagnostic pattern and observing the test fail. The rvalue cases were validated the same way, against the library and not the harness. Reverting the dereference type to `std::iter_reference_t<T>`, the observe-only alternative, makes `reference_or`'s static assertions fail to compile and, with those removed so that they cannot mask the rest, fails the move counts for `value_or` over `optional` and over `expected`, for `or_invoke`, and for `or_construct`. It leaves the smart pointer cases passing, because those copy under both rules. Coverage of `value_or.hpp` is complete on lines, functions, and branches.

Verified `common_reference` results for `reference_or`, showing const propagation:

| nullable (lvalue) | alternative (lvalue) | result |
|---|---|---|
| `optional<int>&` | `int&` | `int&` |
| `const optional<int>&` | `int&` | `const int&` |
| `optional<int>&` | `const int&` | `const int&` |
| `expected<int,int>&` | `int&` | `int&` |
| `int*` | `int&` | `int&` |
| `shared_ptr<int>&` | `int&` | `int&` |
| `optional<string>&` | `string&` | `string&` |

The current implementation, less the portability shims — a polyfill over the compiler builtin for `reference_constructs_from_temporary_v`, and a template parameter order that MSVC accepts:

```
template <class T>
concept nullable = requires(const std::remove_reference_t<T>& t) {
    bool(t);
    *(t);
};

template <class T>
inline constexpr bool enable_borrowed_nullable = false;

template <class T>
concept borrowed_nullable =
    nullable<T> && (std::is_lvalue_reference_v<T> ||
                    enable_borrowed_nullable<std::remove_cvref_t<T>>);

template <class T>
inline constexpr bool enable_borrowed_nullable<T*> = true;
```

```cpp
template <class T>
inline constexpr bool enable_borrowed_nullable<std::optional<T&>> = true;

template <class T, class E>
inline constexpr bool enable_borrowed_nullable<std::expected<T&, E>> = true;

// The type *m yields, carrying the value category of the nullable through
// to its payload.  Exposition only in the wording.
template <class T>
using deref_t = decltype(*std::declval<T>());

template <nullable T, class U,
          class R = std::common_reference_t<deref_t<T>, U&&>>
constexpr auto reference_or(T&& m, U&& u) -> R {
    static_assert(borrowed_nullable<T>);
    static_assert(std::is_reference_v<R>);
    static_assert(!std::reference_constructs_from_temporary_v<R, U>);
    static_assert(
        !std::reference_constructs_from_temporary_v<R, deref_t<T>>);
    return bool(m) ? static_cast<R>(*std::forward<T>(m))
                   : static_cast<R>((U&&)u);
}

template <nullable T, class U,
          class R = std::common_type_t<deref_t<T>, U&&>>
constexpr auto value_or(T&& m, U&& u) -> R {
    return bool(m) ? static_cast<R>(*std::forward<T>(m))
                   : static_cast<R>(std::forward<U>(u));
}

template <nullable T, class I,
          class R = std::common_type_t<deref_t<T>,
                                       std::invoke_result_t<I>>>
constexpr auto or_invoke(T&& m, I&& invocable) -> R {
    return bool(m) ? static_cast<R>(*std::forward<T>(m))
                   : static_cast<R>(std::forward<I>(invocable)());
}

template <class Ret = void, nullable T, class... Args,
          class R = std::conditional_t<std::is_void_v<Ret>,
                        std::remove_cvref_t<deref_t<T>>, Ret>>
constexpr R or_construct(T&& m, Args&&... args) {
    static_assert(!std::is_reference_v<R>);
    static_assert(std::is_constructible_v<R, Args...>);
    static_assert(requires { static_cast<R>(*std::declval<T>()); });
    return bool(m) ? static_cast<R>(*std::forward<T>(m))
                   : R(std::forward<Args>(args)...);
}

template <class Ret = void, nullable T, class E, class... Args,
          class R = std::conditional_t<std::is_void_v<Ret>,
                        std::remove_cvref_t<deref_t<T>>, Ret>>
constexpr R or_construct(T&& m, std::initializer_list<E> il,
                         Args&&... args) {
    static_assert(!std::is_reference_v<R>);
    static_assert(
        std::is_constructible_v<R, std::initializer_list<E>&, Args...>);
    static_assert(requires { static_cast<R>(*std::declval<T>()); });
    return bool(m) ? static_cast<R>(*std::forward<T>(m))
                   : R(il, std::forward<Args>(args)...);
}
```

The Beman reference nullables opt in through explicit integration headers. The core header neither probes the include path for those optional dependencies nor includes them implicitly. This is a packaging boundary

of the reference implementation; the proposed standard-library specializations remain beside their types in the `<optional>` and `<expected>` header synopses below.

The member side is implemented too. `value_or_construct` and `value_or_else` were added to the Beman reference implementations of `optional` [The_Beman_Project_beman_optional] and `expected` [The_Beman_Project_beman_expected], vendored into `beman.free_value_or` and edited in place, on both the primary templates and their reference specializations. `beman.expected` already carries `expected<T&,E>`, `expected<T,E&>`, and `expected<T&,E&>`, so the members run over references now, ahead of any such paper reaching the working draft; the only dependency is standardizing those forms, since implementing them is done. The two libraries' own suites, GoogleTest for `optional` and Catch2 for `expected`, run as part of this project's tests, and cover laziness (a construction counter that never ticks on the engaged path), the `initializer_list` overload, rvalue move-out, constant evaluation, and the reference specializations. All green.

## 7 Proposal

Add to the standard library four constrained free function templates, `value_or`, `reference_or`, `or_invoke`, and `or_construct`, the named concept `nullable`, and the `borrowed_nullable` refinement and customization point, with the semantics described above, targeting C++29. The value-producing functions should be usable on every type modeling `nullable`, including `std::optional`, `std::expected`, raw pointers, `std::shared_ptr`, `std::unique_ptr`, and `std::optional<T&>`. `reference_or` accepts all of them as lvalues, and accepts rvalues only when they model `borrowed_nullable`.

Add the member functions of the other half, as [P3413R0] specified them: `value_or_construct`, with its `initializer_list` overload, and `value_or_else`, on both `std::optional` and `std::expected`, each in `const&` and `&&` form, producing the fallback lazily and in place. Their wording is reproduced below, unchanged in substance from P3413. They apply to `std::optional<T&>` [P2988R12] as well, returning a value; `std::expected<T&,E>` waits on its own paper, as the wording notes. The two feature sets are severable: LEWG may take the free functions, the members, or both, and having one paper means LEWG need not weigh them against each other.

`reference_or` mandates that its computed common reference is a reference type. An rvalue nullable moves its payload in the value-producing functions exactly when dereferencing that rvalue produces an rvalue reference. Thus `optional` and `expected` move their values, while raw and smart pointers copy their referents. Dereferencing an lvalue always would make every owning nullable copy a payload that is going away; this paper instead follows the rvalue's dereference expression (see Design). `or_construct` rejects an explicitly selected reference result type. The conversion dangling guards are *Mandates*, so a dangling call is diagnosed and not removed from the overload set. `nullable` is a public concept. The names are `value_or`, `reference_or`, `or_invoke`, and `or_construct`. The concept stays minimal, so a function pointer models it and is rejected downstream when `common_type` fails, and not at the constraint. The functions do not inhibit ADL. They should be freestanding, and we propose `<optional>` as the header, where users will look for them.

## 8 Wording

Initial draft wording, relative to [N5054], to be completed after LEWG design review.

For a type `T`, let *`deref-t`*`<T>` denote `decltype(*declval<T>())`.

Add to the `<optional>` header synopsis:

```
namespace std {
  template<class T>
    concept nullable = requires(const remove_reference_t<T>& t) {
      bool(t);
      *(t);
    };

  template<class T>
    inline constexpr bool enable_borrowed_nullable = false;
```

```
template<class T>
  concept borrowed_nullable =
    nullable<T> && (is_lvalue_reference_v<T> ||
                    enable_borrowed_nullable<remove_cvref_t<T>>);

template<class T>
  inline constexpr bool enable_borrowed_nullable<T*> = true;
template<class T>
  inline constexpr bool enable_borrowed_nullable<optional<T&>> = true;

template<nullable T, class U>
  constexpr common_reference_t<deref-t<T>, U&&>
    reference_or(T&& m, U&& u);

template<nullable T, class U>
  constexpr common_type_t<deref-t<T>, U&&>
    value_or(T&& m, U&& u);

template<nullable T, class I>
  constexpr common_type_t<deref-t<T>, invoke_result_t<I>>
    or_invoke(T&& m, I&& invocable);

template<class Ret = void, nullable T, class... Args>
  constexpr conditional_t<is_void_v<Ret>,
                          remove_cvref_t<deref-t<T>>, Ret>
    or_construct(T&& m, Args&&... args);

template<class Ret = void, nullable T, class E, class... Args>
  constexpr conditional_t<is_void_v<Ret>,
                          remove_cvref_t<deref-t<T>>, Ret>
    or_construct(T&& m, initializer_list<E> il, Args&&... args);
}
```

Add to the `<expected>` header synopsis:

```
namespace std {
  template<class T, class E>
    inline constexpr bool enable_borrowed_nullable<expected<T&, E>> = true;
}
```

The `expected<T&, E>` specialization depends on standardizing `expected` over reference types.

1 Let `t` be a const lvalue of type `remove_reference_t<T>`. `T` models `nullable` only if:

(1.1) — `bool(t)` is equality-preserving and does not modify `t`;

(1.2) — `*t` is valid only when `bool(t)` is `true`; and

(1.3) — `*t` is equality-preserving and does not modify `t`.

2 The value of `enable_borrowed_nullable<T>` may be set to `true` for a cv-unqualified program-defined type `T` for which destroying an object of type `T` cannot invalidate the result of dereferencing that object. Such a specialization shall be usable in constant expressions and have type `const bool`.

```
template<nullable T, class U>
  constexpr common_reference_t<deref-t<T>, U&&>
    reference_or(T&& m, U&& u);
```

3 Let `R` be `common_reference_t<@deref-t@<T>, U&&>`.

4 *Mandates*: `borrowed_nullable<T>` is `true`, `is_reference_v<R>` is `true`, `reference_constructs_from_temporary_v<R, U>` is `false`, and `reference_constructs_from_temporary_v<R, @deref-t@<T>>` is `false`.

5 *Effects*: Equivalent to: `return bool(m) ? static_cast<R>(*std::forward<T>(m)) : static_cast<R>(std::forward<U>(u));`

```
template<nullable T, class U>
  constexpr common_type_t<deref-t<T>, U&&>
    value_or(T&& m, U&& u);
```

6 Let `R` be `common_type_t<@`*deref-t*`@<T>, U&&>`.

7 *Effects*: Equivalent to: `return bool(m) ? static_cast<R>(*std::forward<T>(m)) : static_cast<R>(std::forward<U>(u));`

```
template<nullable T, class I>
  constexpr common_type_t<deref-t<T>, invoke_result_t<I>>
    or_invoke(T&& m, I&& invocable);
```

8 Let `R` be `common_type_t<@`*deref-t*`@<T>, invoke_result_t<I>>`.

9 *Effects*: Equivalent to: `return bool(m) ? static_cast<R>(*std::forward<T>(m)) : static_cast<R>(std::forward<I>(invocable)());`

```
template<class Ret = void, nullable T, class... Args>
  constexpr conditional_t<is_void_v<Ret>,
                          remove_cvref_t<deref-t<T>>, Ret>
    or_construct(T&& m, Args&&... args);
```

10 Let `R` be `conditional_t<is_void_v<Ret>, remove_cvref_t<@`*deref-t*`@<T>>, Ret>`.

11 *Mandates*: `is_reference_v<R>` is `false`, `is_constructible_v<R, Args...>` is `true`, and the expression `static_cast<R>(*declval<T>())` is well-formed.

12 *Effects*: Equivalent to: `return bool(m) ? static_cast<R>(*std::forward<T>(m)) : R(std::forward<Args>(args)...);`

```
template<class Ret = void, nullable T, class E, class... Args>
  constexpr conditional_t<is_void_v<Ret>,
                          remove_cvref_t<deref-t<T>>, Ret>
    or_construct(T&& m, initializer_list<E> il, Args&&... args);
```

13 Let `R` be `conditional_t<is_void_v<Ret>, remove_cvref_t<@`*deref-t*`@<T>>, Ret>`.

14 *Mandates*: `is_reference_v<R>` is `false`, `is_constructible_v<R, initializer_list<E>&, Args...>` is `true`, and the expression `static_cast<R>(*declval<T>())` is well-formed.

15 *Effects*: Equivalent to: `return bool(m) ? static_cast<R>(*std::forward<T>(m)) : R(il, std::forward<Args>(args)...);`

## 8.1 Member functions, from P3413R0

The following member wording is reproduced from [P3413R0], relative to [N5054], and is proposed jointly with the free functions above. It is purely additive: the existing `value_or` is unchanged. Return types and *Effects* follow P3413; the *Mandates* spell the construction requirement as `is_constructible_v<T, Args...>`, matching the *Effects*.

### 8.1.1 `optional` [optional.optional], [optional.observe]

To the synopsis of `optional` in [optional.optional.general], after the `value_or` declarations, add:

```
template<class... Args> constexpr T value_or_construct(Args&&... args) const&;
template<class... Args> constexpr T value_or_construct(Args&&... args) &&;

template<class U, class... Args>
  constexpr T value_or_construct(initializer_list<U> il, Args&&... args) const&;
template<class U, class... Args>
  constexpr T value_or_construct(initializer_list<U> il, Args&&... args) &&;

template<class F> constexpr T value_or_else(F&& f) const&;
template<class F> constexpr T value_or_else(F&& f) &&;
```

To [optional.observe], add:

```
template<class... Args> constexpr T value_or_construct(Args&&... args) const&;
```

1 *Mandates*: `is_copy_constructible_v<T>` is `true` and `is_constructible_v<T, Args...>` is `true`. *Effects*: Equivalent to: `return has_value() ? **this : T(std::forward<Args>(args)...);`

```
template<class... Args> constexpr T value_or_construct(Args&&... args) &&;
```

2 *Mandates*: `is_move_constructible_v<T>` is `true` and `is_constructible_v<T, Args...>` is `true`. *Effects*: Equivalent to: `return has_value() ? std::move(**this) : T(std::forward<Args>(args)...);`

```
template<class U, class... Args>
  constexpr T value_or_construct(initializer_list<U> il, Args&&... args) const&;
```

3 *Mandates*: `is_copy_constructible_v<T>` is `true` and `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`. *Effects*: Equivalent to: `return has_value() ? **this : T(il, std::forward<Args>(args)...`

```
template<class U, class... Args>
  constexpr T value_or_construct(initializer_list<U> il, Args&&... args) &&;
```

4 *Mandates*: `is_move_constructible_v<T>` is `true` and `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`. *Effects*: Equivalent to: `return has_value() ? std::move(**this) : T(il, std::forward<Args>(args)...);`

```
template<class F> constexpr T value_or_else(F&& f) const&;
```

5 Let `U` be `invoke_result_t<F>`. *Mandates*: `is_copy_constructible_v<T>` is `true` and `is_convertible_v<U, T>` is `true`. *Effects*: Equivalent to: `return has_value() ? **this : std::forward<F>(f)();`

```
template<class F> constexpr T value_or_else(F&& f) &&;
```

6 Let `U` be `invoke_result_t<F>`. *Mandates*: `is_move_constructible_v<T>` is `true` and `is_convertible_v<U, T>` is `true`. *Effects*: Equivalent to: `return has_value() ? std::move(**this) : std::forward<F>(f)();`

7 The reference specialization `optional<T&>` [P2988R12] carries these members too; its wording is given together with `expected<T&,E>` below.

### 8.1.2 `expected` [expected.expected], [expected.object.obs]

To the synopsis of `expected` in [expected.expected.general], after the `value_or` declarations, add:

```
template<class... Args> constexpr T value_or_construct(Args&&... args) const&;
template<class... Args> constexpr T value_or_construct(Args&&... args) &&;

template<class U, class... Args>
  constexpr T value_or_construct(initializer_list<U> il, Args&&... args) const&;
template<class U, class... Args>
  constexpr T value_or_construct(initializer_list<U> il, Args&&... args) &&;

template<class F> constexpr T value_or_else(F&& f) const&;
template<class F> constexpr T value_or_else(F&& f) &&;
```

To [expected.object.obs], add:

```
template<class... Args> constexpr T value_or_construct(Args&&... args) const&;
```

1 *Mandates*: `is_copy_constructible_v<T>` is `true` and `is_constructible_v<T, Args...>` is `true`. *Effects*: Equivalent to: `return has_value() ? **this : T(std::forward<Args>(args)...);`

```
template<class... Args> constexpr T value_or_construct(Args&&... args) &&;
```

2 *Mandates*: `is_move_constructible_v<T>` is `true` and `is_constructible_v<T, Args...>` is `true`. *Effects*: Equivalent to: `return has_value() ? std::move(**this) : T(std::forward<Args>(args)...);`

```
template<class U, class... Args>
  constexpr T value_or_construct(initializer_list<U> il, Args&&... args) const&;
```

3 *Mandates*: `is_copy_constructible_v<T>` is `true` and `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`. *Effects*: Equivalent to: `return has_value() ? **this : T(il, std::forward<Args>(args)...`

```
template<class U, class... Args>
  constexpr T value_or_construct(initializer_list<U> il, Args&&... args) &&;
```

4 *Mandates*: `is_move_constructible_v<T>` is `true` and `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`. *Effects*: Equivalent to: `return has_value() ? std::move(**this) : T(il, std::forward<Args>(args)...);`

```
template<class F> constexpr T value_or_else(F&& f) const&;
```

5 Let `U` be `invoke_result_t<F>`. *Mandates*: `is_copy_constructible_v<T>` is `true` and `is_convertible_v<U, T>` is `true`. *Effects*: Equivalent to: `return has_value() ? **this : T(std::forward<F>(f)());`

```
template<class F> constexpr T value_or_else(F&& f) &&;
```

6 Let `U` be `invoke_result_t<F>`. *Mandates*: `is_move_constructible_v<T>` is `true` and `is_convertible_v<U, T>` is `true`. *Effects*: Equivalent to: `return has_value() ? std::move(**this) : T(std::forward<F>(f)())`

7 `expected<void,E>` gains neither member — there is no value to construct — and no `error_or_construct` or `error_or_else` is proposed, following [P3413R0].

### 8.1.3 Reference specializations, and the expected-over-references dependency

`std::optional<T&>` [P2988R12] is in C++26; `std::expected<T&,E>` is not standardized yet, but is expected to follow. On both, these members return a value rather than a reference — exactly as `value_or` does on those specializations — and are declared `const`, there being no rvalue overload to distinguish. For `optional<T&>` and `expected<T&,E>`, add:

```
template<class... Args>
  constexpr remove_cv_t<T> value_or_construct(Args&&... args) const;
template<class U, class... Args>
  constexpr remove_cv_t<T> value_or_construct(initializer_list<U> il, Args&&... args) const;
template<class F>
  constexpr remove_cv_t<T> value_or_else(F&& f) const;
```

1 Let `X` be `remove_cv_t<T>`, and for `value_or_else` let `U` be `invoke_result_t<F>`. *Mandates*: `is_convertible_v<T&, X>` is `true`; for `value_or_construct`, `is_constructible_v<X, Args...>` (respectively `is_constructible_v<X, initializer_list<U>&, Args...>`) is `true`; for `value_or_else`, `is_convertible_v<U, X>` is `true`. *Effects*: Equivalent to, respectively:

2
```
return has_value() ? **this : X(std::forward<Args>(args)...);
return has_value() ? **this : X(il, std::forward<Args>(args)...);
return has_value() ? **this : std::forward<F>(f)();
```

3 The `optional<T&>` wording above is a plain addition to a C++26 facility; nothing about it is contingent. Only the `expected<T&,E>` case is: it depends on the expected-over-references work, which is not yet in the working draft. Both reference forms are implemented already — the Beman `expected` carries `expected<T&,E>`, `expected<T,E&>`, and `expected<T&,E&>`, and the Beman `optional` the adopted `optional<T&>` — so both forms run today (see Implementation Experience).

## 8.2 Feature test macro

Add a new feature test macro `__cpp_lib_value_or`, set to the date of adoption, for the free functions. For the member functions, bump `__cpp_lib_optional` and `__cpp_lib_expected` to the date of adoption, per [P3413R0]; these are independent of each other and of the `optional<T&>` macro.

# 9 Impact on the standard

A pure library extension. The free functions change no existing class or function. The member functions add `value_or_construct` and `value_or_else` to `optional` and `expected` without altering any existing member, so nothing existing changes meaning; being member function templates, they add nothing to the ABI. The free functions require C++23 library support (`reference_constructs_from_temporary_v`); the members impose no requirement beyond the class they are added to.

# 10 Acknowledgments

Thanks to Peter Sommerlad, co-author of [P2988R12], where the member `value_or` question was fought to a standstill, and to the LEWG reviewers of P1255 and P2988 whose objections shaped these functions. The Tokyo review homework on `value_or` alternatives surveyed the implementation divergence reported here. Thanks to Marc Mutz, whose [P2218R0] originated the member `value_or_construct` and `value_or_else` that [P3413R0] revived, and to Barry Revzin for the wording feedback carried through that paper.

# 11 Document history

— **R1** 2026-10-23 — Unified with [P3413R0] following LEWG's Brno review, which asked for a single proposal. Corentin Jabot joins as co-author, and the members `value_or_construct` and `value_or_else` are folded into the proposal, wording, and implementation. Independently of the unification: an rvalue nullable moves its payload, `nullable` is proposed as a public concept and stated so that const-observability holds, `reference_or`'s dangling guards are corrected and extended to temporary

owning wrappers, reference results are required where appropriate, `or_invoke` forwards its invocable, and the asks to LEWG are cut from nine to zero. See Changes Since Last Version.

— **R0** Brno, 2026-06 — Initial draft, reviewed by LEWG. The free-function follow-on promised in [P2988R12], concluding the line of work begun in [P1255R13].

## References


[Downey_free_value_or] Stephen Downey. beman.free_value_or. https://github.com/steve-downey/free_value_or.

[N5054] Thomas Köppe. N5054: Working draft, programming languages — c++. https://wg21.link/n5054, 7 2026.

[P1255R13] Steve Downey. P1255R13: A view of 0 or 1 elements: views::nullable and a concept to constrain maybes. https://wg21.link/p1255r13, 5 2024.

[P2218R0] Marc Mutz. P2218R0: More flexible optional::value_or(). https://wg21.link/p2218r0, 9 2020.

[P2255R2] Tim Song. P2255R2: A type trait to detect reference binding to temporary. https://wg21.link/p2255r2, 10 2021.

[P2988R12] Steve Downey and Peter Sommerlad. P2988R12: std::optional<t&>. https://wg21.link/p2988r12, 4 2025.

[P3168R2] David Sankel, Marco Foco, Darius Neațu, and Barry Revzin. P3168R2: Give std::optional range support. https://wg21.link/p3168r2, 6 2024.

[P3413R0] Corentin Jabot. P3413R0: A more flexible optional::value_or (else!). https://wg21.link/p3413r0, 10 2024.

[The_Beman_Project_beman_expected] The Beman Project. beman.expected. https://github.com/bemanproject/expected.

[The_Beman_Project_beman_optional] The Beman Project. beman.optional. https://github.com/bemanproject/optional.